

Couche Liaison de données

Couche Liaison de données

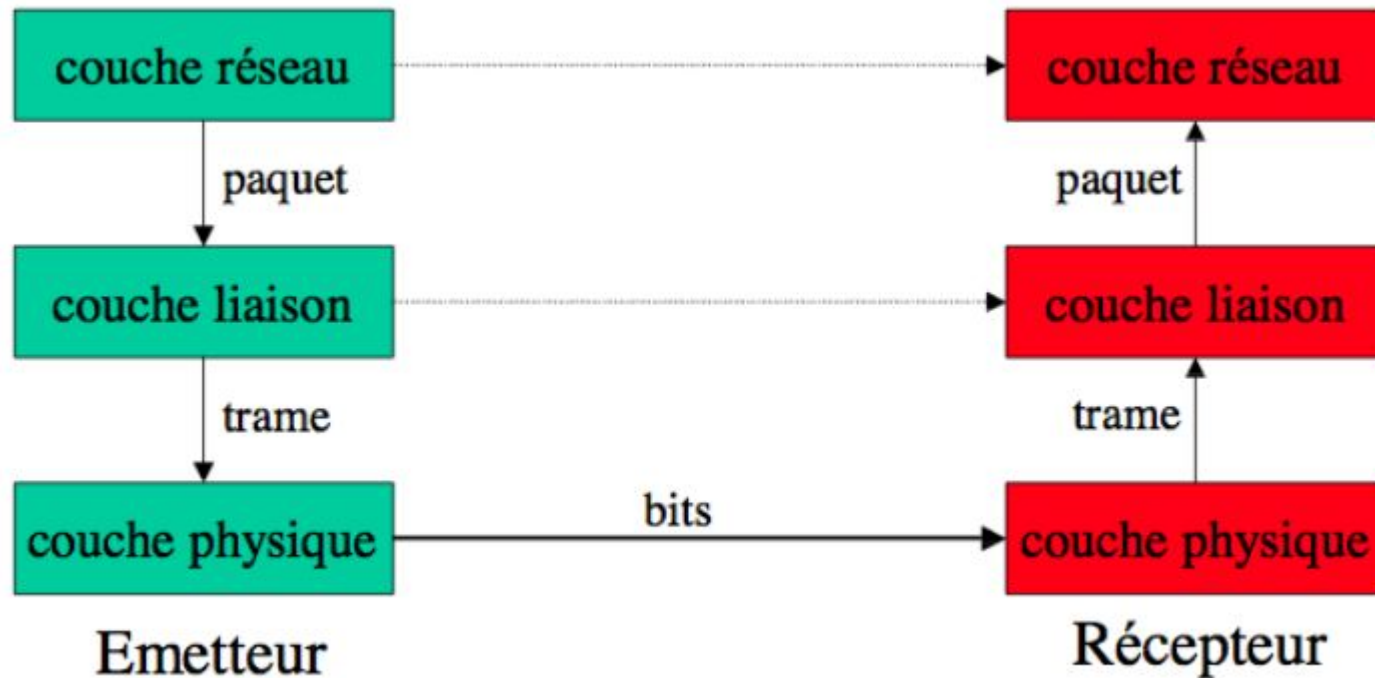
- Rôle

La couche physique permet de transmettre un flot de bits entre deux systèmes distants. La couche liaison (couche n° 2) doit:

1. S'assurer de **la corrections** des bits transmis et **les rassembler** en paquets pour les passer à la couche Réseaux.
2. **Récupérer** des paquets de données de la couche réseau,
3. **Envelopper** les paquets en des trames
4. **Envoyer** les trames une à une à la couche physique.

Couche Liaison de donnée

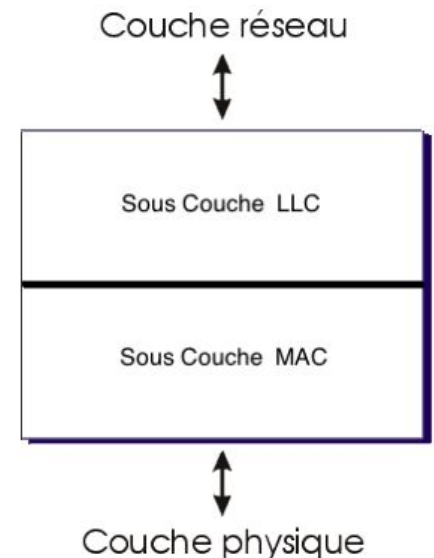
- Rôle



Sous-Couches

La couche liaison de données est divisée en deux sous-couches:

- **Logical Link Control (LLC):**
 1. Gérer les communications entre les dispositifs en contrôlant la synchronisation des trames
 2. le contrôle de flux
 3. la vérification des erreurs.
- **Media Access Control (MAC):**
 1. Le contrôle d'accès à un canal partagé



Sous-couche Logical Link Control (LLC)

Couche liaison

- Le niveau liaison a pour rôle de fiabiliser la transmission physique des données et ainsi de diminuer le taux d'erreurs
- **Fonctionnalités**
 1. Détection des erreurs
 2. Correction ou reprise sur erreurs
 3. Gestion du contrôle de flux
 4. Structure des données en blocs ou trames (PDU : Protocol Data Unit)

Protocole

- Un protocole est un ensemble **de règles** régissant les échanges entre plusieurs entités communicantes.
- Il définit :
 - Format des unités de données échangées et leur type (information ou contrôle)
 - Déroulement de ces échanges dans le temps (timing)

Services de la LLC

- Offre des services classés en trois catégories :
 1. Service sans connexion et sans acquittement
 - Adapté quand le taux d'erreur est faible
 - La correction de transmission est au niveau supérieure
 - **Exemple** : trafic temps réel (parole), Ethernet
 2. Service sans connexion et avec acquittement
 - Acquittement à chaque réception de trame (IEEE 802.11)
 3. Service orienté connexion
 - Établissement préalable d'une connexion

Contrôle de flux

- Le récepteur reçoit des trames de données dans des buffers.
- La taille des buffers est limité.
- Si la vitesse d'arrivée des trames est plus grande que celle du traitement → **Saturation (engorgement) & perte de trames**
- **But du contrôle de flux**
 - Asservir ou ralentir le flux d'émission à la capacité mémoire
 - Deux types de mécanisme :
 1. Trame particulière du récepteur pour stopper l'émetteur
 2. Fenêtre de contrôle de flux permettant l'auto-régulation de l'émetteur

Notion de Trames

- Objectifs:

- Fournir un service à la couche réseau,
- Utiliser le service de la couche physique

- Idée :

- Découpage en trames des trains de bits
- Calcul une somme de contrôle d'erreur pour chaque trame

- Délimitation de trames :

1. Compter les caractères
2. Utiliser des caractères de début et de fin de trame et des caractères de transparence
3. Utiliser des fanions de début et de fin de trame et des bits de transparence

Délimitation de trames

1. Compter les caractères

On utilise un champ dans l'entête de la trame pour indiquer le nombre de caractères de la trame

06	'S'	'U'	'P'	'E'	'R'	03	'L'	'E'	06	'C'	'O'	'U'	'R'	'S'
----	-----	-----	-----	-----	-----	----	-----	-----	----	-----	-----	-----	-----	-----

❖ **Problème:** si erreur du compteur => perte de synchronisation



Délimitation de trames

2. Utiliser des caractères de début et de fin de trame et des caractères de transparence

DLE (Data Link Escape) et ETX (Start of TeXt) en début de trame, et DLE et ETX (End of TeXt) en fin de tram

❖ **Problème:** Données peuvent contenir ces séquences



Délimitation de trames

3. Utilisation des fanions

La trame est délimité par une séquence particulière de bits appelée **fanion** ou **flag**



Pour garantir le bon fonctionnement de cette méthode, **des bits de transparence** sont nécessaires pour qu'une séquence binaire dans la trame ne corresponde accidentellement au fanion.

Si le **fanion** est l'octet **01111110**, un bit de transparence **"0"** est inséré après toute séquence de **cinq 1** successifs dans la trame.

Délimitation de trames

3. Utilisation des fanions

Données :

01011001111110

Trame :

01111110 0101100111111010 01111110

L'avantage des fanions est qu'ils permettent de trouver toujours la **synchronisation** et d'envoyer des trames de **tailles quelconques**

Détection et Correction d'erreurs

Objectifs:

- S'assurer que les données reçues n'ont pas été altérées durant la transmission.

Facteurs de modification le contenu des données

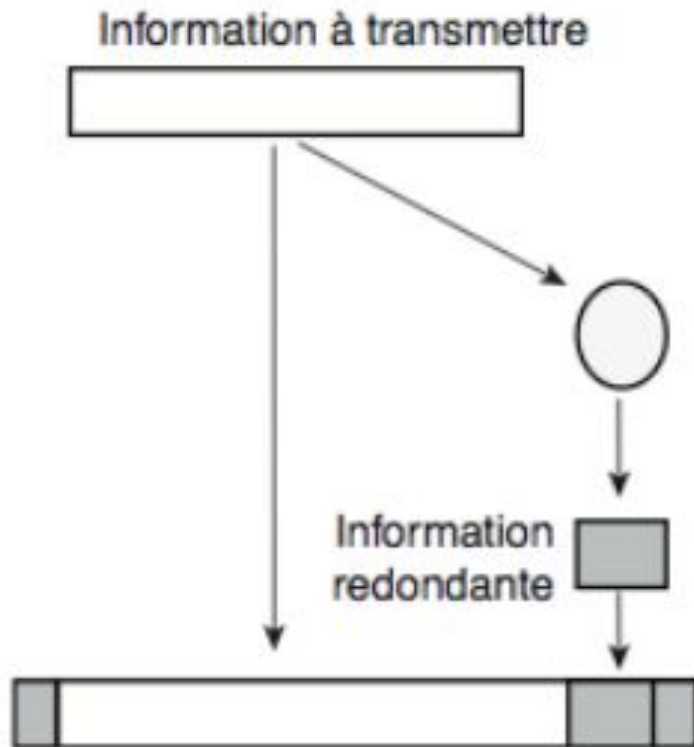
- Les interférences causées par des rayonnements électromagnétiques
- La distorsion des câbles de transmissions.

Idée de protection

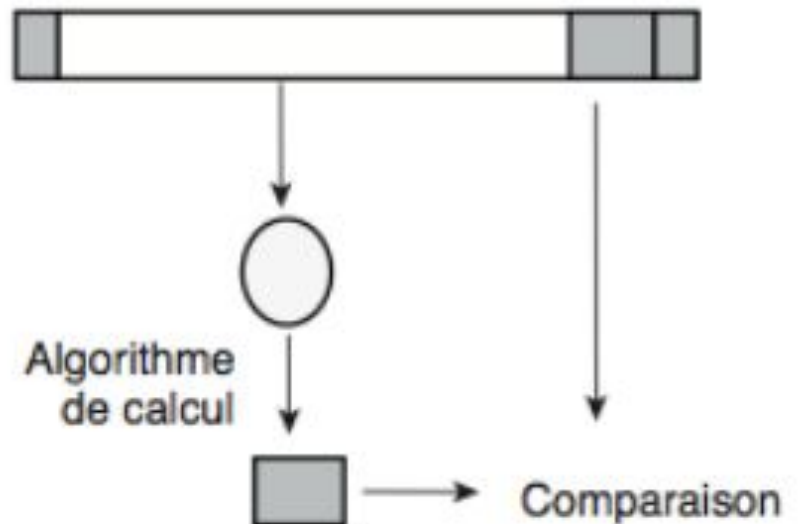
- Redondance de données en ajoutant des bits de contrôle aux bits de données.
- Les bits de contrôle sont calculés, au niveau de l'émetteur, par un algorithme spécifié dans le protocole à partir du bloc de données.
- À la réception, on exécute le même algorithme pour vérifier si la redondance est cohérente. Si c'est le cas, on considère qu'il n'y a pas d'erreur de transmission et l'information reçue est traitée ; sinon, on est certain que l'information est invalide.

Détection et Correction d'erreurs

Calcul à l'émission



Vérification à la réception



Détection et Correction d'erreurs

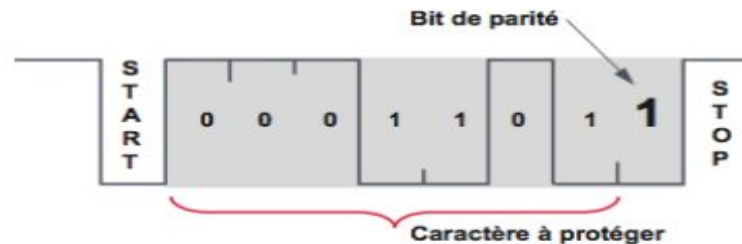
1. Duplication de données

- Le message est envoyé en double exemplaires(détection par répétition).
- Le récepteur sait qu'il y a eu erreur si les exemplaires ne sont pas identiques, il demande alors la retransmission du message.
- Si la même erreur se passe sur les deux exemplaires, l'erreur ne sera pas détectée.
- Si on envoie le message en trois exemplaires, le récepteur pourra même corriger l'erreur en prenant les valeurs des deux copies identiques sans demander la retransmission de l'émetteur

Détection et Correction d'erreurs

2. Code de contrôle de parité Vertical (VRC)

Dans le Vertical Redondancy Check, un bit de parité est ajouté au mot initial pour assurer la parité. Son rendement est faible lorsque k est petit.



Exemple :

Transmission de caracteres utilisant un code de représentation (le code ASCII sur 7 bits)

<u>Lettre</u>	<u>Code ASCII</u>	<u>Mot codé (parité paire)</u>	<u>Mode codé (parité impaire)</u>
E	1010001	10100011	10100010
V	0110101	01101010	01101011
A	1000001	10000010	10000011

Détection et Correction d'erreurs

2. Code de contrôle de parité Vertical (VRC)

<u>Lettre</u>	<u>Code ASCII</u>	<u>Mot codé (parité paire)</u>	<u>Mode codé</u>
E	1 0 1 0 0 0 1	1 0 1 0 0 0 1 1	1 0 1 0 1 0 1 ☐
V	0 1 1 0 1 0 1	0 1 1 0 1 0 1 0	0 1 1 0 1 0 1 ☐
A	1 0 0 0 0 0 1	1 0 0 0 0 0 1 0	1 0 0 0 0 0 1 ☐

<u>Lettre</u>	<u>Code ASCII</u>	<u>Mot codé (parité paire)</u>	<u>Mode codé</u>
E	1 0 1 0 0 0 1	1 0 1 0 0 0 1 1	1 0 1 1 1 0 1 ☐
V	0 1 1 0 1 0 1	0 1 1 0 1 0 1 0	0 1 1 0 1 0 1 ☐
A	1 0 0 0 0 0 1	1 0 0 0 0 0 1 0	1 0 0 0 0 0 1 ☐

Conclusion :

1. Ce code est capable de détecter toutes les erreurs en nombre impair mais il ne détecte pas les erreurs en nombre pair.
2. Il permet de détecter une erreur de parité, mais ne permet pas de la localiser

– 1001011 0
– 1110110 1
– 0010101 1
– 0101000

Détection et Correction d'erreurs

3. Code de contrôle de parité longitudinale

- A chaque bloc de caractère, on ajoute un champ de contrôle supplémentaire (LRC : Longitudinal Redondancy Check)
- La parité longitudinale était initialement utilisée pour les bandes magnétiques pour compléter la détection des erreurs de parité verticale.

Caractère à transmettre	bit de parité	Caractère à transmettre	bit de parité	...	Caractère LRC	bit de parité
-------------------------------	---------------------	-------------------------------	---------------------	-----	------------------	---------------------

Détection et Correction d'erreurs

4. Code de contrôle de parité double

- Le bloc de données est disposé sous une forme matricielle ($k = a \cdot b$).
- On applique la parité sur chaque ligne et chaque colonne. On obtient une matrice ($a + 1, b + 1$).
- Un caractère le LRC est ajouté au bloc transmis.
- Chaque bit du caractère LRC correspond à la parité des bits de chaque caractère de même rang :
 - le premier bit du LRC est la parité de tous les 1^{er} bits de chaque caractère,
 - le second de tous les 2^e bits... Le caractère ainsi constitué est ajouté au message.
- Le LRC est lui-même protégé par un bit de parité (VRC).

Détection et Correction d'erreurs

4. Code de contrôle de parité double

	Bit 0	Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	VRC
H	0	0	0	1	0	0	1	0
E	1	0	1	0	0	0	1	1
L	0	0	1	1	0	0	1	1
L	0	0	1	1	0	0	1	1
O	1	1	1	1	0	0	1	1
LRC	0	1	0	0	0	0	1	0

1001000	0	1000101	1	1001100	1	1001100	1	1001111	1	1000010	0
H		E		L		L		O		LRC	

Détection et Correction d'erreurs

5. Distance de Hamming

La distance de Hamming entre deux mots x et y est le nombre de bits sur lesquels ils diffèrent.

$$d(00010, 01010) = 1$$

$$d(00011010, 00001011) = 2$$

On note $B(x, k)$ la boule de centre x et de rayon k : c'est l'ensemble des mots qui diffèrent de x par moins de k bits.

Détection et Correction d'erreurs

5. Distance de Hamming

- Un code détecte k erreurs si pour tout $x \neq y$, $B(x, k)$ ne contient pas y .
- Un code corrige k erreurs si pour tout $x \neq y$, $B(x, k)$ et $B(y, k)$ ne s'intersectent pas.
- Un code détecte k erreurs si la distance entre deux mots de code est supérieure à $k + 1$
- Un code corrige k erreurs si la distance entre deux mots de code est supérieure à $2k + 1$
- Les bits de contrôle sont situés aux puissances de 2.
- Le bit de contrôle en position 2^i est la parité de tous les bits ayant 2^i dans leur décomposition en binaire.

Détection et Correction d'erreurs

6. Codes Polynomiaux (CRC (Cyclic Redundancy Check))

- le CRC est la méthode la plus utilisée pour détecter des erreurs groupées.
- Avant la transmission, on ajoute des bits de contrôle. Si des erreurs sont détectées à la réception, il faut retransmettre le message.
- Dans ce code, une information de n bits est considérée comme la liste des coefficients binaires d'un polynôme de n termes, donc de degré $n-1$

Exemple :

$$- 1101 \rightarrow x^3 + x^2 + 1$$

$$- 110001 \rightarrow x^5 + x^4 + 1$$

Détection et Correction d'erreurs

6. Codes Polynomiaux (CRC (Cyclic Redundancy Check))

Pour calculer les bits de contrôle, on effectue un certain nombre d'opérations avec ces polynômes à coefficients binaires. Toutes ces opérations sont effectuées modulo 2.

C'est ainsi que, dans les additions et dans les soustractions, on ne tient pas compte de la retenue : Toute addition et toute soustraction sont donc identiques à une opération XOR.

Détection et Correction d'erreurs

6. Codes Polynomiaux (CRC (Cyclic Redundancy Check))

$$\begin{array}{r} 1 0 0 1 1 0 1 1 \\ + 1 1 0 0 1 0 1 0 \\ \hline = 0 1 0 1 0 0 0 1 \end{array}$$

$$\begin{array}{r} 1 1 1 1 0 0 0 0 \\ - 1 0 1 0 0 1 1 0 \\ \hline = 0 1 0 1 0 1 1 0 \end{array}$$

Détection et Correction d'erreurs

6. Codes Polynomiaux (CRC (Cyclic Redundancy Check))

La source et la destination choisissent un même polynôme $G(x)$ dit générateur car il est utilisé pour générer les bits de contrôle (Checksum).

L'algorithme pour calculer le message à envoyer est le suivant. Soit $M(x)$ le polynôme correspondant au message original, et soit r le degré du polynôme générateur $G(x)$ choisi :

- Multiplier $M(x)$ par x^r , ce qui revient à ajouter r zéros à la fin du message original
- Effectuer la division suivante modulo 2 :

$$\frac{M(x)x^r}{G(x)} = Q(x) + R(x)$$

- Le quotient $Q(x)$ est ignoré. Le reste $R(x)$ (Checksum) contient r bits (degré du reste $r - 1$). On effectue alors la soustraction modulo 2 :

$$M(x).x^r - R(x) = T(x)$$

Le polynôme $T(x)$ est le polynôme cyclique : c'est le message prêt à être envoyé. Le polynôme cyclique est un multiple du polynôme générateur $T(x) = Q(x).G(x)$

A la réception, on effectue la division suivante :

$$\frac{T(x)}{G(x)}$$

- Si le reste = 0, il n'y a pas d'erreur
- Si le reste $\neq 0$, il y a erreur, donc on doit retransmettre

En choisissant judicieusement $G(x)$, on peut détecter toute erreur sur 1 bit, 2 bits consécutifs, une séquence de n bits et au-delà de n bits avec une très grande probabilité.